

Elio Govea

Senior Software Engineer · [Vaisala](#)

Systems engineer in networking and embedded Linux, now building cross-platform systems on Linux and Windows — from home routers to aviation weather platforms. ICPC World Finalist.

Helsinki, Finland eliogovea1993@gmail.com [github.com/eliogovea](#) [LinkedIn](#) Spanish (native) · English (professional)

Summary

Eight years of production C++ across networking, embedded Linux and cross-platform delivery. At [Vaisala](#), extending [AviMet](#) — an aviation weather platform running in 170+ countries — onto new Linux targets, and building the simulation tooling its modernization is verified against. Before that, four and a half years at [F-Secure](#) on the Sense SDK: home-router security embedded by 200+ service-provider partners and shipped across x86, ARM and MIPS, where I traced a production cost problem to its root and cut **monthly CDN spend by 80%**.

Skills

LANGUAGES	C++ (11–20), C, Python, Bash, PowerShell, TypeScript / JavaScript
NETWORKING	netlink, netfilter, iptables, nfqueue, nflog, conntrack, BPF, Unix domain sockets
PROTOCOLS	IPv4 / IPv6, TCP, UDP, DHCP, DNS, HTTP / HTTPS, TLS, QUIC
PLATFORMS	Embedded Linux, Buildroot, OpenWrt, Alma Linux, x86 / ARM / MIPS, Windows
BUILD & CI	CMake, GNU Make, Cross-compilation, Git, Docker, LXC, Jenkins, AWS CI, Valgrind, Visual Studio
FRAMEWORKS	Qt, Robot Framework, OpenGL
CONCEPTS	Algorithms & data structures, Performance optimization, Static & dynamic analysis, Debugging & profiling

Experience

[Vaisala](#) · Senior Software Engineer Feb 2025 – Present

Vantaa, Finland · [AviMet](#) aviation weather platform — deployed across 170+ countries, 2,500+ systems delivered

- Made AviMet's behavior provable to the standard **ICAO Annex 3** aeronautical meteorology demands — one programmable reference driving both the legacy and modernized systems into arbitrary states from simulated sensor data, spanning weather scenarios, sensor input and communication protocols, so correctness can be verified repeatably and equivalence testing becomes possible.
- Replaced GUI-bound tooling with an interface developers, testers and AI-assisted workflows can script — what previously meant working through an extensive configuration checklist by hand now runs from defaults almost immediately, turning scattered team knowledge into executable documentation.
- Led AviMet's expansion onto **Alma Linux** — extending the C++/Qt/CMake build matrix and toolchain to a second Linux distribution, and implementing Linux-side license enforcement against Vaisala's existing licensing system.
- Built tooling for a new cloud entitlements system, designed for reuse across Vaisala products beyond AviMet.
- Hardened a long-lived C++ codebase spanning Windows and Linux — root-caused a 32-bit Windows leak that presented as a null-pointer dereference, and a leak triggered only by audio-device disconnect, which on a remote machine happens only once the RDP session ends: it never reproduced while anyone was watching (Valgrind, Visual Studio).

[F-Secure](#) Sep 2020 – Feb 2025

Helsinki, Finland · Poznań, Poland · Sense SDK — C++ home-router security embedded by 200+ service-provider partners · 8–10 developers, ~20 with integration and test

Senior Software Engineer

May 2023 – Feb 2025

- Cut **monthly CDN spend by 80%** with no user-visible impact. Root-caused it to router-side behavior driving excessive traffic; a router update would have taken months to reach deployed devices, so I shipped a backend-side mitigation in days instead, then landed the router-side fix for future releases to close the cause permanently.
- Co-designed and implemented cloud-driven dynamic traffic-rule enforcement, letting F-Secure push new security policy to deployed routers without firmware updates.

Software Engineer

Sep 2020 – Apr 2023

- Built core Sense SDK functionality — URL filtering and parental controls (forcing safe-mode browser engines), traffic-statistics collection, device recognition and fingerprinting — across IPv4/IPv6, DHCP, TCP/UDP, DNS and HTTP/HTTPS, using Linux networking interfaces (netlink, netfilter, iptables, nfqueue, nflog, conntrack, BPF) against cloud back-end services.
- Built the embedded Linux clients consuming those cloud services, the Unix-domain-socket IPC between clients and services, and reworked the security-event notification path.
- Owned the cross-platform build system (CMake + GNU Make) for fast, reproducible builds across x86, ARM and MIPS toolchains, plus SDK integrations for reference platforms that shortened partner demos and onboarding.
- Designed the Jenkins/AWS CI pipelines and the quality gates behind them — coverage, static and dynamic analysis — producing builds for every supported architecture and reference platform.
- Built the test infrastructure in Python and Robot Framework, including recovery paths for OS- and cloud-service-level failures.

Universidad de Pinar del Río · Software Engineer

Jan 2018 – Aug 2020

Pinar del Río, Cuba

- Built real-time C++/Qt applications for vibration-signal processing and IMU-based object position tracking — embedded sensor input over serial (Arduino), Linux, OpenGL visualization.
- Served as ICPC judge for the Caribbean region and coached a team to the ICPC Latin American Finals, alongside teaching C++, Python, Arduino, Linux and Raspberry Pi.

Education

Universidad de Pinar del Río · B.Sc. Telecommunications and Electronics Engineering

Sep 2012 – Jul 2017

Pinar del Río, Cuba · GPA 4.95 / 5.00 · ICPC World Finalist

Achievements

- **Honorable Mention** — 2018 [ICPC World Finals](#), Beijing (140-team final, from ~50,000 regional competitors)
- **2nd Place, Silver Medal** — 2016 ICPC Latin American Finals
- **4th Place** — 2015 ICPC Latin American Finals
- **Round 3, Top 1000** — 2019 Google Code Jam (of ~50,000 entrants — top ~2%)

Selected projects

DNS library 2022–2024 — Wire format parsed and generated from scratch.

C++ project template 2021–2026 — Identical builds on any machine.

Baseball Limits 2D 2024–2026 — Pareto frontiers over 150 years of MLB data, shipped client-side. [Live demo ↗](#)

Router-network Docker testbed 2024 — Client → router → server, in containers, no hardware.

Automatic layout for a growing DAG 2019–2026 — Construction order fixes one axis; forces settle the rest. [Live demo ↗](#)

[More projects →](#)